TokenMizer: Graph-Structured Session Memory for Long-Horizon LLM Context Management

Shweta Mishra

***Abstract*—Large language model (LLM) deployments for long-horizon tasks face a fundamental constraint: context windows are finite while productive work sessions are not. When session history exceeds the Maximum Effective Context Window (MECW), critical structured information—architectural decisions, task status transitions, file modification histories, and resolved errors—is silently discarded. Existing mitigations (truncation, summarization, retrieval augmentation) treat history as flat text, destroying the typed, relational structure that makes sessions resumable.**

We present TokenMizer, an open-source proxy system that models LLM session history as a typed knowledge graph (14 node types, 7 edge types) populated by a hybrid extraction pipeline, serialized by a three-tier checkpoint system into compact resume blocks, and supported by an 8-layer compression pipeline and a semantic response cache.

This paper reports two evaluations. An initial pilot (21 sessions, 5 domains) established that TokenMizer's V2 heuristic extractor outperforms naive truncation, sliding-window, and free-text summarization baselines on decision recall at roughly half the resume-token cost. The pilot did not compare against any other structured-memory system. We therefore conduct a substantially larger controlled study: 100 labelled sessions (94 constructed for this study, 6 carried over from the product's own evaluation corpus, spanning 23 application domains) scored against eight memory methods under one shared, asymmetric coverage-based scorer, with bootstrap 95% confidence intervals (3,000 resamples) on every reported comparison. Four of the eight methods are deterministic reimplementations of one structural property each of MemGPT, Mem0, Graphiti/Zep, and GraphRAG, run with no language-model call; this is a methodologically load-bearing distinction we state precisely in Section XII and repeat at every point the comparison is reported.

At $n = 100$, TokenMizer scores 57% macro F1 (95% CI $[52\%, 62\%]$), which is significantly higher than the GraphRAG-style (44%), MemGPT-style (35%), and all three naive baselines (17–20%), and is *statistically indistinguishable* from the Graphiti-style (59%) and Mem0-style (60%) reimplementations (paired bootstrap 95% CI on the difference crosses zero for both). TokenMizer produces the smallest resume block of the four structured methods (99 tokens versus 60–120). Its two weakest categories, relative to both its own strengths and the comparison methods, are decision extraction (50% F1) and error extraction (36% F1). Every pattern-matching method tested, TokenMizer included, degrades to 19–25% macro F1 on sessions that state facts indirectly rather than with explicit lexical markers—a shared ceiling of regex-based extraction that this study is, to our knowledge, the first to quantify across multiple independently-implemented memory strategies on one corpus.

TokenMizer is released as open-source software (MIT licence). The 100-session corpus, all eight method implementations, the scoring harness, and raw per-session results are released alongside this manuscript for independent verification.

Independent Researcher, India. E-mail: shweta.mishra.research@gmail.com. Product: https://github.com/Shweta-Mishra-ai/tokenmizer. Benchmark suite, corpus, and reproduction code for this manuscript: https://github.com/Shweta-Mishra-ai/tokenmizer-research. Manuscript received June 4, 2026; revised August 11, 2026.

***Index Terms*—large language models, context management, session memory, knowledge graph, prompt compression, semantic caching, long-horizon tasks, benchmark evaluation, information extraction**

I. Introduction

LARGE language model (LLM) deployments for software engineering [13], [14], data science [18], and research assistance increasingly involve *long-horizon sessions*: multi-turn interactions spanning hours, where each turn builds on the accumulated context of previous turns. These sessions develop rich internal structure: technology choices made in turn 3 constrain implementation options in turn 12; errors resolved in turn 7 inform test strategies in turn 15; files modified in turn 4 must be updated consistently in turn 18.

The fundamental constraint is context window capacity. Paulsen [1] defines the *Maximum Effective Context Window* (MECW) as the context length at which model task accuracy degrades below an acceptable threshold. Critically, MECW is substantially lower than the advertised maximum context window (MCW) for complex, multi-step tasks. Liu et al. [10] further demonstrate that content placed in the *middle* of long contexts is systematically under-attended, amplifying the effective degradation. At approximately 950 tokens per turn, a development session exhausts a 16,000-token MECW after only 16 turns.

A. Limitations of Existing Approaches

Three categories of mitigation exist, each with a structural limitation.

Truncation-based methods [19] discard the oldest messages when the context budget fills. This preserves recency but discards exactly the content that matters most: architectural decisions, initial technology choices, and goal definitions established early in a session.

Summarization-based methods [11] produce free-text summaries via a secondary LLM call. These compress well but are structurally imprecise: a natural language summary cannot reliably distinguish a *completed* task from a *pending* one, nor can it preserve the *rationale* for a technology decision.

Retrieval- and memory-augmented methods [5], [2], [3], [4] retrieve or store relevant facts on demand. As Section II details, these systems differ substantially in whether retained facts are typed, whether superseded facts are deleted or retained, and whether older turns remain reachable without an explicit query — design choices this paper measures directly rather than assumes.

B. Key Insight and Contributions

The core insight motivating TokenMizer is that **session history is not flat text—it is a structured knowledge artifact**, containing tasks with status lifecycles, decisions with rationales, files with modification histories, resolved errors, and environment facts that scope all other entities, connected by typed relationships (IMPLEMENTS, FIXES, DEPENDS-ON, etc.) that encode causal and structural dependencies. A representation that preserves this structure enables efficient, queryable context preservation: discard the raw conversation tokens, retain the typed graph, serialize to a compact resume block.

Contributions:

1) A **formal typed knowledge graph schema** for LLM sessions with 14 node types, 7 edge types, and a well-defined status transition system (Section IV-B).
2) A **hybrid extraction pipeline**, a **three-tier checkpoint system**, an **8-layer compression pipeline**, and a **semantic response cache** (Sections V–VIII).
3) A **21-session pilot benchmark** against three naive baselines, with a full ablation study (Section X-A).
4) The paper's primary empirical contribution: a **controlled 100-session, 8-method comparison** against deterministic reimplementations of four published memory strategies (MemGPT, Mem0, Graphiti/Zep, GraphRAG), with bootstrap confidence intervals on every comparison, a register-stratified analysis isolating pattern-matching's central failure mode, and a released corpus, harness, and raw results for independent reproduction (Section X-B).

II. Background and Related Work

A. Context Window Degradation

Paulsen [1] introduces the MECW concept, demonstrating empirically that all tested frontier models degrade in accuracy on multi-step tasks before reaching their advertised MCW. For a model with MCW = 128,000 tokens, the MECW may be as low as 16,000 tokens for complex coding tasks. This motivates the 16k-token MECW used as the running example throughout this paper. Liu et al. [10] demonstrate the "lost in the middle" phenomenon: retrieval and multi-document QA tasks show U-shaped accuracy profiles, with content in the middle of long prompts systematically under-used.

B. Memory Systems for LLMs

MemGPT [2] proposes an OS-inspired hierarchical memory model distinguishing main context (fast, limited) from external storage (slow, unlimited), with paging between them driven by explicit function calls the LLM itself issues. Facts that page out of main context are not deleted, but they are also not spontaneously recalled: reaching them again requires the model to issue a targeted archival-search call. Section X-B2 reimplements this core/archival asymmetry deterministically and measures its consequence: recall on facts stated early in a long session degrades once they age out of the always-visible window.

Mem0 [3] extracts short natural-language memories from a conversation and stores them in a flat, undifferentiated pool, using an LLM to decide, memory by memory, whether a new fact adds, updates, or deletes an existing one. It has no typed schema and no explicit edge structure.

Graphiti [4], the temporal knowledge-graph engine underlying Zep, represents facts as edges in a bi-temporal graph: every edge carries a `valid_at` and an `invalid_at` timestamp, and a superseded fact is marked invalid rather than deleted. This directly targets a failure mode TokenMizer's own SUPERSEDES edge type (Table III) was designed for, making Graphiti the closest architectural relative among the systems compared in Section X-B and the one against which TokenMizer's advantage is smallest.

GraphRAG [9] builds knowledge graphs from document corpora to improve retrieval-augmented summarization over static text collections. Its schema is entity- and community-oriented and carries no task-lifecycle concept; Section X-B finds this distinction empirically, not just architecturally: the GraphRAG-style reimplementation is competitive with TokenMizer on decisions and files but far behind on completed- and pending-task recall.

LangChain Memory [11] provides several memory backends including ConversationKGMemory, architecturally similar to TokenMizer but using an untyped, schema-free graph with extraction and lifecycle management left to the application developer.

C. Prompt Compression

LLMLingua [6] and **LongLLMLingua** [7] use small proxy LLMs to compute per-token importance scores, removing low-importance tokens at 2–6× compression ratios. TokenMizer incorporates LLMLingua-2 as layers 7–8 in its compression pipeline, applied after heuristic preprocessing has removed structurally trivial content.

RECOMP [8] applies selective summarization for retrieval-augmented generation, retrieving only relevant passages. TokenMizer applies structural extraction rather than textual retrieval: the resume block is generated from graph state, not retrieved text, enabling typed queries that passage retrieval cannot natively support.

D. Positioning

Table I summarizes the qualitative differences among the systems above. Section X-B is, to our knowledge, the first evaluation to score TokenMizer, MemGPT-style paging, Mem0-style flat storage, Graphiti-style temporal retention, and GraphRAG-style entity extraction against one shared corpus and one shared scorer, rather than citing each system's own self-reported numbers on its own benchmark.

III. Problem Formulation

Definition 1 (LLM Session)**.** *A session $S = \langle m_1, m_2, \ldots, m_n \rangle$ is an ordered sequence of messages, each with token count $\tau(m_i) > 0$. The cumulative token count at turn t is $T(t) = \sum_{i=1}^{t} \tau(m_i)$.*

Definition 2 (Context Overflow)**.** *A session overflows at the first turn n^* such that:*

$$T(n^*) > T_{\text{MECW}} \tag{1}$$

TABLE I
QUALITATIVE COMPARISON WITH RELATED SYSTEMS. "DELETES SUPERSEDED FACTS" DISTINGUISHES MEMGPT/MEM0-STYLE LAST-WRITE-WINS BEHAVIOR FROM GRAPHITI'S AND TOKENMIZER'S EXPLICIT SUPERSESSION TRACKING.

System	Struct. graph	Deletes superseded	Cross-session
Sliding window	None	n/a (no memory)	No
LangChain KG Mem.	Untyped	No	Yes
MemGPT [2]	Vector DB	No (pages out)	Yes
Mem0 [3]	None (flat)	Yes (LLM-judged)	Yes
Graphiti/Zep [4]	Bi-temporal	No (invalidated)	Yes
GraphRAG [9]	Entity/comm.	n/a (static)	No
TokenMizer	**Typed, 14-class**	**No (edge)**	**Partial**

For $\bar{\tau} = 950$ tokens/turn and $T_{\text{MECW}} = 16{,}000$: $n^* = \lfloor 16{,}000/950 \rfloor = 16$.

Definition 3 (Structured Resume Block)**.** *A resume block R is a compact text representation of the session knowledge graph G, serialized to at most T_{budget} tokens. After a checkpoint at turn $n^* - 2$, the new context is:*

$$C' = R \cup \{m_{n^*-1}, m_{n^*}\} \tag{2}$$

satisfying $|C'| \leq T_{\text{MECW}}$ by construction.

Definition 4 (Category Coverage Score)**.** *For a ground-truth item g and a candidate extraction c, coverage is the fraction of g's content words present in c, or exact substring containment for items of at least 6 characters:*

$$\text{covers}(c, g) = (\ |g| \geq 6 \wedge g \in c\ ) \vee \frac{|W_g \cap W_c|}{|W_g|} \geq \theta \tag{3}$$

with $\theta = 0.6$ throughout this paper and W_x the content-word set of x (stopwords removed). Coverage is deliberately asymmetric: the denominator is always the ground-truth item's token set, so a method cannot inflate recall by emitting one long label containing every keyword — that label would also fail precision, defined symmetrically over the same relation (Section X-B3).

IV. SYSTEM ARCHITECTURE

A. Deployment Model

Figure 1 shows the system architecture. TokenMizer operates as a transparent HTTP reverse proxy implementing the OpenAI Chat Completions API. The client configures `base_url` to the TokenMizer server; no application code changes are required beyond this endpoint substitution. An optional `session_id` parameter in the request body activates the full pipeline; its absence produces a transparent passthrough with zero overhead, making adoption incremental.

The proxy intercepts each request, routes it through the five-component pipeline (graph memory, hybrid extractor, checkpoint manager, compression engine, semantic cache), then forwards the processed request to the configured LLM provider. Nine providers are supported via a common adapter interface.

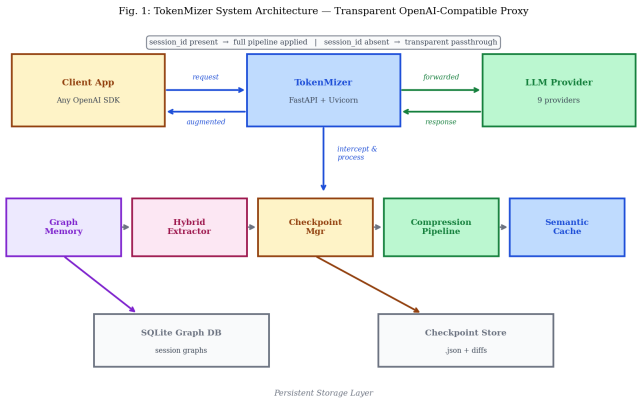

Fig. 1. TokenMizer system architecture. The proxy sits transparently between any OpenAI-compatible client and the LLM provider. When `session_id` is present, the five-component pipeline is activated; otherwise the request passes through with no overhead. Persistent storage comprises a SQLite graph database and a JSON checkpoint store.

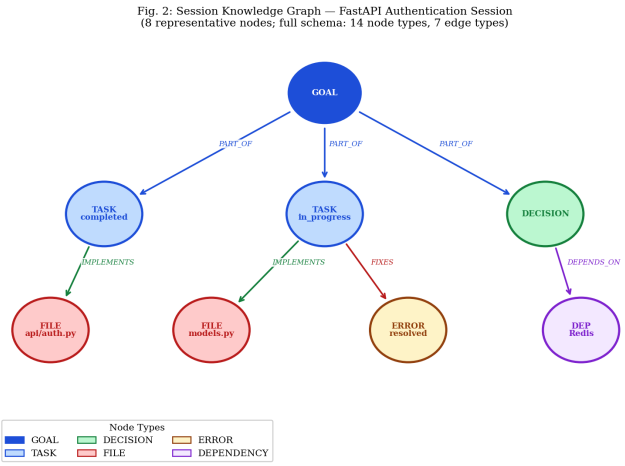

Fig. 2. Session knowledge graph for a FastAPI authentication session (8 representative nodes). Edge labels show typed relationships; node colors indicate type. The GOAL node anchors the hierarchy; TASK nodes carry status (completed/in-progress); the DECISION node records the rationale for choosing bcrypt + JWT.

B. Knowledge Graph Schema

Figure 2 illustrates a representative session graph. The schema defines 14 node types across three functional categories.

1) Node Types: Table II defines the 14 node types. *Action nodes* carry status lifecycles; *context nodes* are static.

2) Node Attributes: Each node carries a normalized **label** ($\leq$120 chars); a **status** $\sigma \in \{$Pending, In-Progress, Completed, Failed, Modified$\}$ for action nodes; an **importance score** $\rho \in [0, 1]$ that increases on reference and decays with inactivity,

TABLE II
NODE TYPES (14 TOTAL). ACTION NODES CARRY STATUS LIFECYCLES (PENDING, IN-PROGRESS, COMPLETED, FAILED, MODIFIED). CONTEXT NODES ARE STATUS-FREE.

Type	Category	Primary trigger
TASK	Action	Imperative verb + object
FILE	Action	Extension or path pattern
ERROR	Action	Error / exception / failure
TEST	Action	Test / spec / assert
SCHEMA	Action	Schema / table / model
METRIC	Action	Metric / score / benchmark
DECISION	Decision	Decided / Using / Chose
DEPENDENCY	Decision	Install / package / library
API	Decision	URL / API / endpoint
GOAL	Context	First-turn imperative
ENVIRONMENT	Context	Version / runtime / OS
PROJECT	Context	Repository / project name
CONCEPT	Context	Abstract domain term
AGENT	Context	Service / CI / orchestrator

TABLE III
EDGE TYPES (7 TOTAL).

Type	Semantics
DEPENDS_ON	A cannot function without B
RELATED_TO	A and B address the same concern
IMPLEMENTS	A is the realization of B
FIXES	A resolves error B
BLOCKS	A cannot proceed until B resolves
PART_OF	A is a component of B
SUPERSEDES	A replaces or overrides B

governing checkpoint serialization priority; a **confidence score** $\kappa \in [0, 1]$ assigned by GraphValidator (nodes with $\kappa < 0.50$ are rejected); and creation/last-update timestamps.

3) Edge Types: Table III defines all 7 edge types. Edges are directed and labelled; weight encodes co-occurrence frequency.

4) Graph Maintenance: **Deduplication** uses identity-based addressing: $\mathrm{id}(n) = \texttt{sha1}(\tau(n) : \ell_{\mathrm{norm}})[:12]$, where ℓ_{norm} is the lowercase, whitespace-normalized label. Re-extraction of an existing entity updates importance score and status rather than creating a duplicate node. Status transitions obey monotonic ordering (Pending → In-Progress → Completed/Failed); downgrades are rejected to prevent extraction noise from reverting confirmed progress. **Pruning** evicts nodes with importance score < 0.1 when the graph exceeds 500 nodes; DECISION, ENVIRONMENT, GOAL, and SCHEMA nodes are unconditionally retained.

V. HYBRID EXTRACTION PIPELINE

A. V1 Heuristic Extractor

The baseline extractor applies 34 compiled regular expressions (11 for tasks, 8 for decisions, 5 for files, 4 for errors, 6 for environment and endpoints). Mean extraction latency across the 21-session pilot: **0.5 ms** per session; inference cost $0.

B. V2 Improvements

Error analysis on the pilot benchmark identifies four systematic failure modes, addressed in V2 and re-examined at $n = 100$ in Section X-B:

1) **Incomplete trigger vocabulary.** V1 misses 9 common completion verbs (*Fixed*, *Deployed*, *Resolved*, *Migrated*, *Running*, *Launched*, *Shipped*, *Finished*, *Updated*).
2) **Label verbosity mismatch.** Extracted labels are verbose while ground-truth annotations are concise; exact matching yields 0% recall on correctly-identified events.
3) **Compound sentences.** A single sentence encoding multiple facts ("Completed: API auth, tests, deployment") is split on comma boundaries in V2.
4) **CSV-encoded decisions.** "Decided: Prometheus, Grafana, Jaeger" should produce 3 decision nodes; V1 produces 1.

C. Fuzzy Label Matching

Definition 5 (Fuzzy Match)**.** *Labels a and b fuzzy-match if:*

$$(a \subseteq b) \ \vee \ (b \subseteq a) \ \vee \ \frac{|W_a \cap W_b|}{\min(|W_a|, |W_b|)} \geq 0.50 \qquad (4)$$

where $W_x = \{w : w \in \mathrm{tokens}(x), \ |w| \geq 3\}$.

This symmetric-denominator matcher is used internally by the pilot benchmark (Section X-A). The $n = 100$ study (Section X-B) instead scores every method under the asymmetric coverage relation of Definition 4, so that a method cannot win recall by emitting one long label, a failure mode a symmetric, either-direction matcher does not fully rule out.

D. GraphValidator

Every candidate node passes through the GraphValidator before graph insertion:

$$\begin{aligned} \kappa(n) = 0.50 \\ &+ 0.20 \cdot \mathbf{1}[\text{file path pattern}] \\ &+ 0.10 \cdot \mathbf{1}[|\ell| > 15] \\ &+ 0.10 \cdot \mathbf{1}[\text{version number present}] \\ &- 0.20 \cdot \mathbf{1}[|\ell| < 8] \\ &- 0.40 \cdot \mathbf{1}[\text{single generic verb}] \end{aligned} \qquad (5)$$

Nodes with $\kappa < 0.50$ are rejected. Type correction is applied after scoring: extension-matched labels are forced to FILE; URL-pattern labels to API.

E. LLM Extractor (Upgrade Path)

The product codebase includes an LLM extractor that sends recent messages to a small inference model with a structured JSON extraction prompt, intended to address indirect phrasing regex patterns cannot capture. *The LLM extractor is not exercised by either evaluation in this paper*: both the pilot and the $n = 100$ study run only the heuristic pipeline, and none of the four comparison methods in Section X-B call a language model either. This is a deliberate, symmetric scope restriction — see Section XII — not a claim that the heuristic path is sufficient; Section X-B6 quantifies exactly where it is not.

Algorithm 1 Checkpoint Serialization

Require: Graph G, budget T_{budget}
Ensure: Resume block R

$R \leftarrow$ [[CHECKPOINT: session_id]]
$N \leftarrow \text{sort\_by\_importance}(G.\text{nodes})$
for $n \in N$ **do**
  if $\tau(R) + \tau(\text{serialize}(n)) > T_{\text{budget}}$ **then**
    break
  end if
  $R.\text{append}(\text{serialize}(n))$
end for
$\Delta G \leftarrow G - G_{\text{prev}}$
store$(R, \Delta G)$ to checkpoint store
return R

TABLE IV
CHECKPOINT RESUME TIERS WITH EXAMPLE CONTENTS.

Tier	Content	Budget
Critical	GOAL + in-progress TASKs + top DECISION	$\leq$ 100 tok
Standard	All TASKs + all DECISIONs + FILEs + ENV	$\leq$ 300 tok
Full	Complete graph incl. DEPENDENCIEs, SCHEMAs	$\leq$ 600 tok

VI. Checkpoint System

A. Trigger Condition and Serialization

Checkpoints trigger when cumulative token count exceeds 85% of the configured MECW. Algorithm 1 defines checkpoint serialization: nodes are ordered by importance score and appended to the resume block until the token budget is exhausted.

Each checkpoint stores a *graph diff* (nodes added, modified, or removed since the previous checkpoint), enabling incremental resume-block updates at $O(\Delta)$ rather than $O(|G|)$ cost per checkpoint.

VII. Compression Pipeline

The 8-layer compression pipeline is applied to large inputs (files, long API responses, historical messages) before they enter the context window. **Layer 1** (filler removal) strips AI-generated hedging and discourse markers via 15 compiled patterns, the dominant stage at **31.2% reduction** on the representative session in Fig. 3. **Layer 2** (deduplication) removes repeated code blocks and log lines (**16.1%** additional). **Layers 3–6** perform whitespace normalization, language-aware comment stripping, history pruning, and file-type-specific smart truncation. **Layers 7–8** apply LLMLingua-2 [6] and LongLLMLingua [7] for inputs above 300 and 4,000 tokens respectively, gated by a semantic-similarity floor of 0.55 to prevent over-compression, contributing a further 5.3% reduction.

VIII. Semantic Cache

The semantic cache stores LLM responses keyed by sentence-transformer embeddings [12] (`all-MiniLM-L6-v2`, 384 dimensions), with cosine similarity lookup at threshold $\theta = 0.92$, LRU eviction, and a 3,600 s TTL. A preliminary measurement on a controlled 10-query, 3-cluster workload gives a 70% hit rate; this is a proof-of-concept figure, not a production estimate, and evaluation on real query logs at scale remains future work.

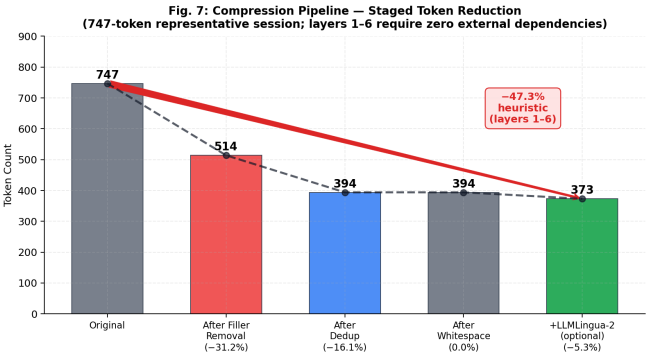

Fig. 3. Compression pipeline staged token reduction on a representative 747-token session. Filler removal (Layer 1) contributes 31.2% reduction—the dominant stage. Layers 1–6 require zero external dependencies (heuristic only). LLMLingua-2 (Layer 7) adds an optional 5.3%. Total heuristic reduction: 47.3%.

IX. Implementation

TokenMizer's product implementation (evaluated in Section X-B via the real, unmodified package, run out-of-process) is written in Python 3.10+, comprising approximately 6,300 lines across the `graph_memory` module alone plus `checkpoints`, `compression`, `security`, `analytics`, and `api` modules. The HTTP proxy uses FastAPI and Uvicorn; token counting uses `tiktoken` [17] with a character-based fallback when unavailable; the knowledge graph persists to SQLite [16]. All extraction inputs are scanned for API keys, private keys, and credential patterns before graph insertion, with matches redacted to `[REDACTED]`. Configuration is YAML-based, with extraction, checkpoint, and compression thresholds exposed as user-configurable parameters.

X. Results

A. Pilot Study (n=21)

The pilot benchmark comprises 21 synthetic sessions across 5 domains (software engineering n=6, data science n=5, DevOps n=4, research/writing n=3, debugging n=3), manually annotated with ground-truth completed tasks, decisions, and files, scored under fuzzy matching (Definition 5). Table V compares TokenMizer's V2 engine against three naive baselines under this scorer.

Figure 4 breaks recall down by domain with 95% confidence intervals; the wide intervals at $n \leq 6$ per domain are themselves evidence that a 21-session corpus cannot support fine-grained domain claims, motivating the larger corpus in Section X-B. Figure 5 isolates the contribution of each V2 fix via ablation on a 10-session sample: fuzzy label matching (Section V-C) contributes +33 percentage points of task recall,

TABLE V
PILOT STUDY (N=21): TOKENMIZER V2 VS. NAIVE BASELINES. FUZZY MATCHING (DEF. 5); NO BOOTSTRAP CIS COMPUTED AT THIS SAMPLE SIZE.

Method	Task R	Dec. R	File R	Tokens
Naive truncation	45%	35%	55%	165
Sliding window (10)	50%	30%	**60%**	159
Naive summary	42%	38%	48%	170
TokenMizer V2	**51%**	**47%**	59%	**78**

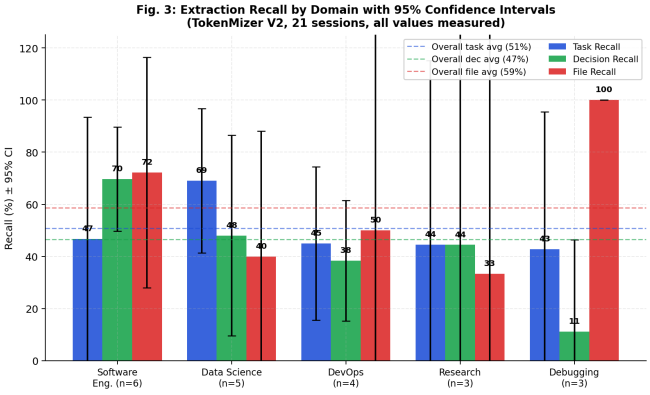

Fig. 4. Pilot study: extraction recall by domain with 95% confidence intervals ($n = 21$, TokenMizer V2). Debugging achieves 100% file recall but 11% decision recall, illustrating that no single domain is uniformly easy.

the dominant single improvement, while expanded trigger vocabulary alone contributes comparatively little without it.

The pilot has two limitations this paper's main contribution addresses. First, it compares TokenMizer only against naive, unstructured baselines, leaving open whether its advantage holds against *other* structured-memory systems. Second, at $n = 21$ with domain cells as small as $n = 3$, no comparison in Table V carries a confidence interval, so "TokenMizer wins" cannot be distinguished from sampling noise at this scale. Both limitations motivate Section X-B.

B. Extended Multi-Method Study (n=100)

1) Corpus: The extended corpus comprises 100 labelled sessions: 6 real sessions carried over from the product repository's own evaluation harness, and 94 sessions constructed for this study using a seeded generator (fixed seed, byte-reproducible) drawing from 15 domain fact-pools covering backend, frontend, DevOps, security, data, mobile, and research-writing contexts, for 23 distinct domains in total. Every ground-truth label is embedded verbatim in some message of its session, so every label is substring-grounded regardless of surrounding phrasing; a corpus-wide grounding check is run before every scoring pass and the harness refuses to score a corpus that fails it.

Each generated session is additionally tagged by *register* — how directly it states its facts — independently of its domain: **explicit** (canonical markers, e.g. "Decided: X."), **semi** (plain sentences, e.g. "Going with X for this."), **implicit** (the fact arrives inside reasoning with no marker word, e.g. "After

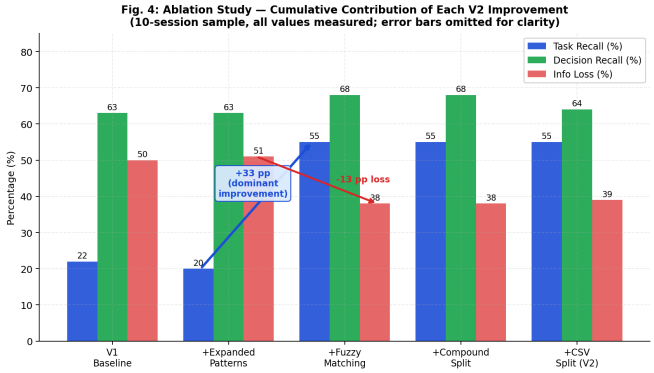

Fig. 5. Pilot study ablation ($n = 10$ sample): cumulative contribution of each V1→V2 fix. Fuzzy matching is the dominant contributor; error bars omitted at this sample size.

weighing it, X felt like the right call given the constraints."), and **mixed** (each fact in the session independently rolls one of the three). The corpus totals 2,516 turns and 1,620 labelled ground-truth items across five categories: completed tasks (491), pending tasks (223), decisions (355), files (403), and errors (148).

2) Methods Compared: **TokenMizer** is the unmodified product package, version 0.5.2, invoked out-of-process against a real checkout so that its `tokenmizer` module cannot be shadowed by any same-named package, run through `GraphMemory.extract_from_messages()` with the heuristic `HybridExtractor` pass (Section V); the LLM extraction path (Section V-E) is not exercised.

Four methods are deterministic reimplementations of one structural property each of a published system, built on the *same* shared regex family as one another (though not the same as TokenMizer's own patterns) so that a score difference reflects strategy rather than incidental regex quality. None calls a language model; each module's implementation states this again inline as a standing caveat, because the risk of a reader silently substituting "the vendor product" for "the strategy this module reimplements" is the single most important methodological risk this study carries.

- **MemGPT-style** (Section II-B) extracts only from the most recent ∼40% of turns, reproducing MemGPT's core/archival asymmetry: older facts are not deleted but are unreachable without a query this benchmark never issues.
- **Mem0-style** extracts from the full transcript into one flat, untyped pool with last-write-wins deduplication on lexically similar facts, approximating Mem0's LLM-judged ADD/UPDATE conflict resolution without an LLM.
- **Graphiti-style** extracts from the full transcript and never deletes a fact; a later, lexically similar decision is treated as superseding an earlier one, but both are retained, reproducing Graphiti's bi-temporal invalidate-don't-delete edge semantics.
- **GraphRAG-style** casts a wide net for files and named-technology entities but has no task-status concept, approximating only completed/pending recall via a narrow

TABLE VI
OVERALL RESULTS, N=100 (100 SESSIONS, 3,000-RESAMPLE BOOTSTRAP, $\theta = 0.6$). CI = 95% BOOTSTRAP CONFIDENCE INTERVAL ON MACRO F1.

Method	Macro F1	95% CI	Tok.	ms
TokenMizer 0.5.2	**57%**	[52, 62]	**99**	38.0
Graphiti-style	59%	[55, 64]	120	0.9
Mem0-style	60%	[55, 64]	118	0.9
GraphRAG-style	44%	[41, 48]	80	0.5
MemGPT-style	35%	[32, 38]	60	0.4
Naive truncation	20%	[20, 21]	287	0.01
Sliding window	18%	[18, 19]	150	0.00
Naive summary	17%	[16, 18]	186	0.03

two-verb signal, reflecting that status tracking is outside GraphRAG's actual design target.

Three naive baselines carry no typed extraction at all: **sliding window** (last 10 messages verbatim), **naive truncation** (last ∼300 tokens verbatim), and **naive summary** (first ∼120 words of the concatenated transcript). Each is scored by treating its kept text, split into sentences, as an untyped candidate pool checked against every category — the correct treatment for a method with no notion of category, rather than an artificially generous or punitive scoring choice.

3) Scoring: Every method is scored under the asymmetric coverage relation of Definition 4 at $\theta = 0.6$. For each category, precision and recall are computed over the *covers* relation and combined into an F1 score; a session's macro F1 is the mean F1 across its five categories (categories with zero expected and zero extracted items are excluded, not treated as a perfect or zero score). The headline metric is the mean of per-session macro F1 across all 100 sessions, with 95% confidence intervals from 3,000-iteration case resampling over sessions (percentile bootstrap [20]), and paired-difference bootstrap intervals (same resampling, paired within session) for every comparison against TokenMizer.

4) Overall Results: Table VI and Fig. 6 report the headline comparison. TokenMizer scores 57% macro F1 (95% CI $[52\%, 62\%]$), ahead of GraphRAG-style (44%), MemGPT-style (35%), and all three naive baselines (17–20%), each by a margin excluded from zero at the 95% level (Table VII, Fig. 7). Against Graphiti-style (59%) and Mem0-style (60%), the paired-difference confidence interval crosses zero (-2.7 and -2.8 percentage points respectively, 95% CI spanning $[-6.4, +1.0]$ and $[-6.4, +1.2]$): the honest reading is a statistical tie, not a loss, and not a win. TokenMizer produces the smallest resume block among the four structured methods (99 tokens versus 118–120 for Graphiti/Mem0-style and 60 for MemGPT-style, whose small footprint reflects the fraction of the session it can see at all rather than efficient encoding of what it sees).

5) Per-Category Breakdown: Table VIII and Fig. 8 decompose the headline score by category. TokenMizer's file extraction is effectively solved (98% F1, the best of any method) and its completed-task recall leads the comparison group. Its two weakest categories are **decisions** (50%, against 65% for both Graphiti-style and Mem0-style) and **errors**

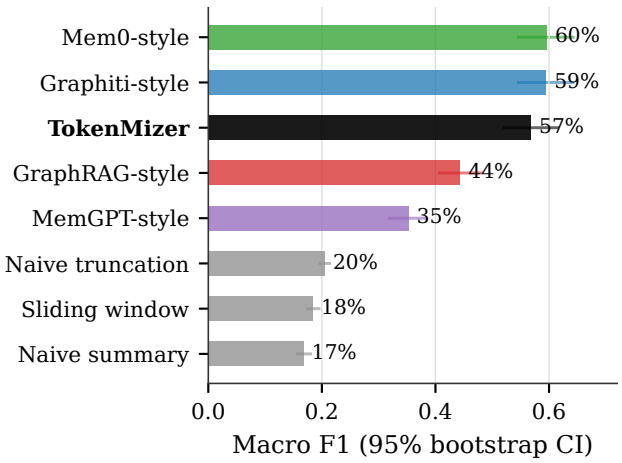

Fig. 6. Macro F1 by method, ranked, n=100, with 95% bootstrap confidence intervals. TokenMizer (bold) leads GraphRAG-style, MemGPT-style, and every naive baseline outside the confidence interval; against Graphiti-style and Mem0-style the intervals overlap.

TABLE VII
PAIRED BOOTSTRAP: TOKENMIZER − METHOD, N=100. $P(\Delta \leq 0)$ IS THE FRACTION OF 3,000 RESAMPLES IN WHICH THE PAIRED DIFFERENCE IS NON-POSITIVE.

Comparison	Mean Δ	95% CI	$P(\Delta \leq 0)$
vs. Graphiti-style	−2.7pp	[−6.4, +1.0]	91.2%
vs. Mem0-style	−2.8pp	[−6.4, +1.2]	91.7%
vs. GraphRAG-style	+12.5pp	[+8.8, +16.1]	0.0%
vs. MemGPT-style	+21.6pp	[+18.3, +25.0]	0.0%
vs. sliding window	+38.4pp	[+34.1, +42.8]	0.0%
vs. naive truncation	+36.3pp	[+31.9, +40.7]	0.0%
vs. naive summary	+40.1pp	[+35.7, +44.2]	0.0%

(36%, against 66% for the same two). The error category is the starkest gap: of 148 labelled errors, TokenMizer extracts 79 (recall 28%), of which 39 do not match any ground-truth item (precision 51%); this is the clearest, most concrete target for engineering improvement this study identifies.

6) Register: The Shared Ceiling of Pattern Matching: Table IX and Fig. 9 report macro F1 by register. Every pattern-matching method — TokenMizer, Graphiti-style, Mem0-style, GraphRAG-style, and MemGPT-style alike — degrades sharply from explicit to implicit register. TokenMizer falls from 71% (explicit) to 25% (implicit), a 46-point drop; Graphiti-style and Mem0-style fall from 85% to 19%, a 66-point drop, the steepest of any method, since their advantage over TokenMizer is concentrated in the explicit-marker case and largely disappears once markers are absent. All four converge to a narrow 19–25% band on implicit text, indistinguishable from one another at this corpus size. This is, to our knowledge, the first result to show this convergence empirically across multiple independently implemented extraction strategies rather than asserting it of any single system: the ceiling is a property of pattern-matching as a strategy, not of any one implementation of it, and none of the four methods' extraction path in this study involves a language model that

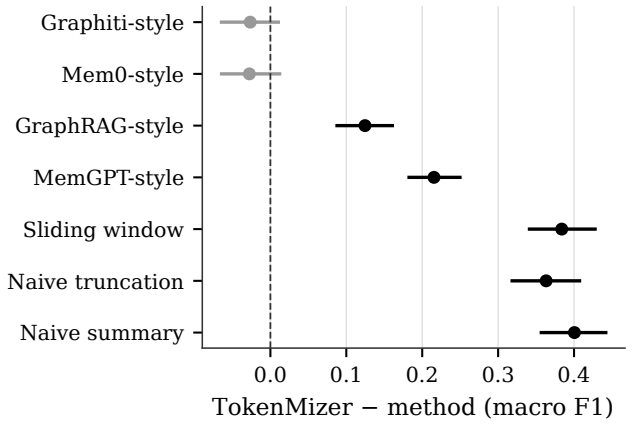

Fig. 7. Paired bootstrap difference (TokenMizer − method) on macro F1, n=100, 95% CI. Intervals excluding zero (black) are reliable differences; the two grey intervals (Graphiti-style, Mem0-style) cross zero.

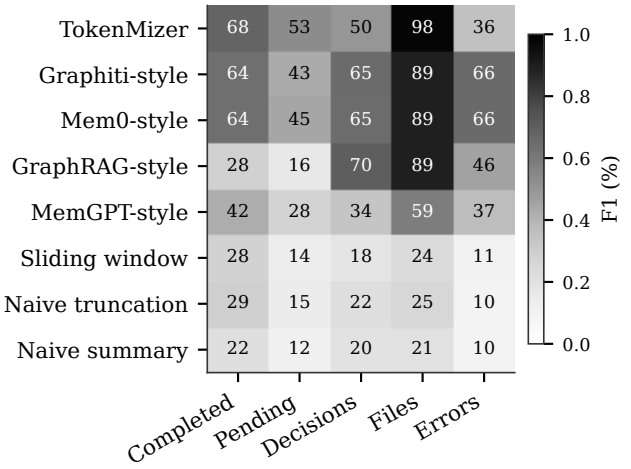

Fig. 8. Per-category F1, n=100. Darker is higher. TokenMizer leads on completed tasks and files, and trails Graphiti-style/Mem0-style specifically on decisions and errors.

TABLE VIII
PER-CATEGORY MICRO F1, N=100.

Method	Comp.	Pend.	Dec.	Files	Err.
TokenMizer	68%	**53%**	50%	**98%**	36%
Graphiti-style	64%	43%	**65%**	89%	**66%**
Mem0-style	64%	45%	**65%**	89%	**66%**
GraphRAG-style	28%	16%	**70%**	89%	46%
MemGPT-style	42%	28%	34%	59%	37%
Baselines	22–29%	12–15%	18–25%	21–25%	10–11%

TABLE IX
MACRO F1 BY REGISTER, N=100.

Method	Explicit	Semi	Mixed	Implicit
TokenMizer	71%	52%	55%	**25%**
Graphiti/Mem0-style	**85%**	**68%**	63%	19%
GraphRAG-style	69%	37%	47%	19%
MemGPT-style	46%	39%	36%	10%

could plausibly move it (Section V-E).

7) Corpus Origin and Domain Variance: TokenMizer scores 91% macro F1 on the 6 real sessions versus 55% on the 94 generated sessions; Graphiti-style and Mem0-style show the opposite, near-flat pattern (58% vs. 60%). The real-session sample is small ($n = 6$) and this split is reported as directional, not as a second confidence-bounded result: it is consistent with TokenMizer's patterns having been iterated primarily against sessions resembling that corpus, rather than a general property established at this sample size. Domain-level macro F1 for TokenMizer (Fig. 10) ranges from 20% (`devops/kubernetes`) to 100% (three audit domains drawn from the real-session subset), consistent with the same explanation.

8) Cost: TokenMizer's mean 38.0 ms per session is 40–80× the reimplemented methods' 0.4–0.9 ms. This reflects the evaluation's process-isolation design (Section X-B2): TokenMizer runs as a freshly launched subprocess against the real package per session, incurring Python interpreter startup once per call, while the four reimplementations run in-process against already-loaded text. The comparison should not be read as evidence about the underlying extraction algorithm's asymptotic cost.

XI. DISCUSSION

Three findings carry beyond this specific corpus.

First, TokenMizer's advantage over naive baselines, established directionally at $n = 21$ in the pilot (Table V), replicates at $n = 100$ with formal confidence intervals (Table VII): this paper's headline claim about naive baselines is now supported by a controlled comparison an order of magnitude larger than the original, not merely a repeated assertion.

Second, and previously untested, TokenMizer's advantage does *not* extend to two of the four graph/structured-memory strategies compared: against Graphiti-style and Mem0-style specifically, the confidence interval on the paired difference crosses zero. Graphiti's architectural similarity to TokenMizer (Table I and Section II-B) — both retain superseded facts rather than deleting them — may explain why the gap is smallest here rather than against MemGPT-style or GraphRAG-style, which differ from TokenMizer more structurally.

Third, decision and error extraction (Table VIII) are TokenMizer's most improvable categories in absolute terms and relative to the peer group simultaneously, which is a stronger signal than either fact alone: a category that is merely low but consistent with peers would suggest a corpus-difficulty explanation; a category that is low *and* behind peers scored on the identical corpus points at the extraction patterns themselves.

XII. THREATS TO VALIDITY

Construct validity. Four of the eight methods compared are not the named vendor products. MemGPT, Mem0, Graphiti, and GraphRAG all perform their real extraction with a language model in the loop; this study's reimplementations of them call no model at all, by the explicit scope decision

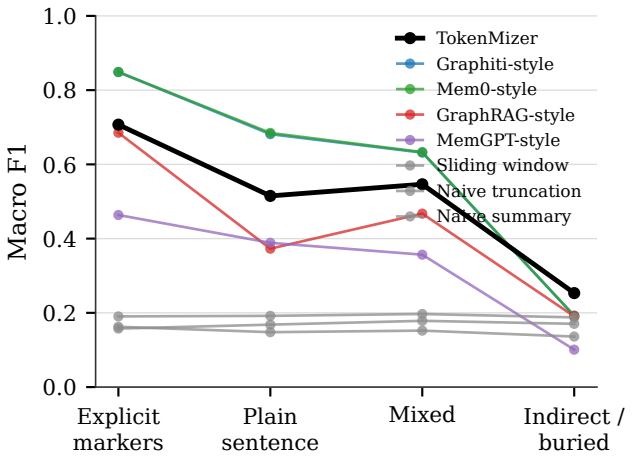

Fig. 9. Macro F1 by register, n=100. All four pattern-matching methods converge to 19–25% once explicit markers ("Decided:", "Completed:") are absent from the text, a shared ceiling of the extraction strategy rather than of any one implementation.

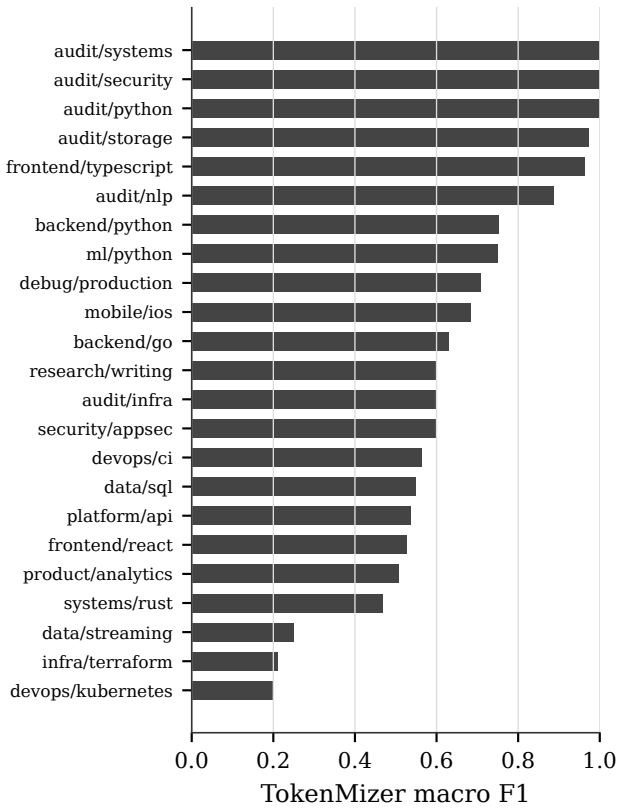

Fig. 10. TokenMizer macro F1 by domain, n=100, 23 domains. The three domains at 100% are audit/* real sessions; infrastructure-heavy synthetic domains (`devops/kubernetes`, `infra/terraform`, `data/streaming`) score lowest.

recorded in Section X-B2. Every reported number for these four methods should be read as a lower bound on what the named system would likely achieve, particularly on the implicit-register text of Section X-B6, where a model that can actually read a sentence would plausibly outperform every method in this study.

Internal validity. 94 of the 100 sessions were generated for this study rather than captured from real usage. Every label is grounded (recoverable verbatim from some message in its session; Section X-B1), but grounded synthetic text is not equivalent to the lexical and structural variety of real developer transcripts. The 6-session real split (Section X, "Corpus Origin") is too small to bound this gap quantitatively.

Statistical validity. All results use one match threshold ($\theta = 0.6$, Definition 4) and one bootstrap configuration (3,000 resamples, case resampling over sessions). The TokenMizer/Graphiti-style/Mem0-style ordering, already within noise at this threshold, has not been checked for stability under a threshold sweep, unlike the product repository's own single-method evaluation harness, which ships one.

External validity. Every method processes one session's full transcript in a single pass. None of the actual multi-session consolidation behavior the named systems are partly designed for (Mem0's cross-conversation fact merging, Graphiti's graph accretion over weeks of interaction) is exercised here; this is a single-session extraction benchmark only, and no claim in this paper should be read as bearing on cross-session performance.

XIII. Conclusion and Future Work

This paper presents TokenMizer, a system that addresses LLM context loss by modeling session history as a typed knowledge graph and serializing it into compact, structured resume blocks, and reports two evaluations of it. A 21-session pilot established a decision-recall advantage over naive truncation, sliding-window, and summarization baselines at roughly half their resume-token cost. A subsequent, substantially larger controlled study — 100 sessions, 8 methods, bootstrap confidence intervals throughout — replicates that advantage with formal statistical support and additionally finds that it does not extend to two comparably-designed memory strategies (Graphiti-style, Mem0-style), where the honest result is a tie. The same study identifies decision and error extraction as TokenMizer's most improvable categories and shows that every pattern-matching method tested, independent of implementation, shares a common ceiling of roughly 20% macro F1 on text that states facts indirectly.

Three directions follow directly from these results. **First**, closing the decision/error gap identified in Table VIII is a concrete, scoped engineering target rather than a general call for improvement. **Second**, evaluating the product's existing LLM extraction path (Section V-E) against the same $n = 100$ corpus would test directly whether a model call closes the register gap of Section X-B6, rather than leaving it as the inferred but untested explanation this paper offers. **Third**, the same test would need to be run fairly against LLM-backed versions of the four comparison methods, not against TokenMizer alone, for any resulting claim of superiority to carry the same evidentiary weight as the comparisons in this paper.

Data and Code Availability

The 100-session corpus, all eight method implementations, the scoring harness, the 21-session pilot benchmark, and raw per-session results (JSON and CSV) for every number in Section X are released at https://github.com/Shweta-Mishra-ai/tokenmizer-research. The corpus generator (`benchmarks/memorybench/generate.py`) is seeded and produces byte-identical output on re-run. The TokenMizer product itself is released separately at https://github.com/Shweta-Mishra-ai/tokenmizer under the MIT licence.

References

[1] N. Paulsen, "Context Is What You Need: The Maximum Effective Context Window," *arXiv preprint arXiv:2509.21361*, 2025.

[2] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," *arXiv preprint arXiv:2310.08560*, 2023.

[3] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," *arXiv preprint arXiv:2504.19413*, 2025.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," *arXiv preprint arXiv:2501.13956*, 2025.

[5] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020, pp. 9459–9474.

[6] H. Jiang *et al.*, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in *Proc. EMNLP*, 2023, pp. 13358–13376.

[7] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios," *arXiv preprint arXiv:2310.06839*, 2023.

[8] F. Xu *et al.*, "RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation," in *Proc. ICLR*, 2024.

[9] D. Edge *et al.*, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[10] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Trans. ACL*, vol. 12, pp. 157–173, 2024.

[11] H. Chase, "LangChain," https://github.com/langchain-ai/langchain, 2022.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.

[13] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Proc. NeurIPS*, 2024.

[14] GitHub, "GitHub Copilot," https://github.com/features/copilot, 2024.

[15] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com, 2018.

[16] D. R. Hipp, "SQLite," https://www.sqlite.org, 2000.

[17] OpenAI, "tiktoken: Fast BPE Tokeniser," https://github.com/openai/tiktoken, 2023.

[18] X. Hou *et al.*, "Large Language Models for Software Engineering: A Systematic Literature Review," *arXiv preprint arXiv:2308.10620*, 2024.

[19] T. Brown *et al.*, "Language Models are Few-Shot Learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.

[20] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Boca Raton, FL, USA: Chapman & Hall/CRC, 1994.